EP News Screener

AI-Powered Stock Announcement Screening System
for Indian Equity Markets (BSE / NSE)

Tech Stack: Python 3.12 | SQLite + FTS5 | ChromaDB | Claude AI (Haiku + Opus)
36 files | ~5,000 lines | 7 packages | 2 AI agents | 3 search methods

---

1,268 announcements processed | *May 4 - June 23, 2026*

1. Problem Statement

Every trading day, BSE and NSE publish 700-4000 corporate announcements. Most are noise (AGM notices, newspaper copies, record dates). Hidden in this flood are EP (Episodic Pivot) catalysts - large orders, earnings surprises, acquisitions, capacity commissioning - that can drive 10-50% stock moves.

Manual screening takes 2-3 hours per day and misses cross-referencing opportunities. This system automates the entire pipeline from raw CSV to AI-powered analysis.

2. Solution Overview

End-to-end pipeline that ingests raw exchange CSVs, filters noise (80% dropped), scores catalysts (1-13), enriches with sector/materiality/novelty metadata, stores in dual databases (SQL + vector), and answers natural language questions via AI agents.

Architecture Flow

```
CSV (4000 rows)
  |-- Screener: filter + score --> 200-750 candidates
  |-- Enrichment: sector tags + materiality + novelty
  |-- Storage: SQLite (structured) + ChromaDB (semantic)
  |-- Retrieval: BM25 + Vector + RRF fusion + Reranker
  |-- Agent: Claude explains ranked candidates
  --> User gets actionable EP watchlist
```

3. Technology Stack

Layer	Technology	Why This Choice
Language	Python 3.12	Data + ML ecosystem
SQL Database	SQLite + FTS5	Zero-config, built-in BM25 search
Vector Database	ChromaDB	all-MiniLM-L6-v2 via ONNX, cosine sim
LLM (Agent)	Claude Haiku 4.5	Fast, cheap ($1/M tokens)
LLM (PDF Extract)	Claude Opus 4.7	Accuracy-first for financial tables
Search Fusion	RRF (k=60)	Merges BM25 keyword + vector semantic
Reranking	Proxy (Jaccard+score)	No extra model dependency
PDF Download	urllib (stdlib)	Browser UA, retry, SSL, 10MB cap

4. Package Architecture (7 Packages, 36 Files)

Package 1: screener/ -- CSV Screening Pipeline

FilterConfig: 28 drop subjects, 16 keep subjects, 6 compiled regexes
CsvParser.parse(): reads CSV, maps BSE/NSE column variants, 7 datetime formats
AnnouncementFilter.should_keep(ann): first gate, drops ~80% noise
ScoringEngine.score(ann): dispatch table of 15 subject handlers, scores 1-13
EPScreener.run(min_score, conn): orchestrator - filter > score > enrich > sort

Package 2: enrichment/ -- Metadata Enrichment (Zero LLM)

SectorTagger.tag(): 13 sector labels via regex (defence, railway, solar, pharma...)
MaterialityExtractor.extract(): order value (Rs.Cr/Lakh/Mn/USD), relative_size (tiny/small/medium/large/mega), catalyst_type, government_linked, export_component
NoveltyDetector.detect(): Jaccard similarity vs recent DB rows, keyword signals

Package 3: storage/ -- Dual Database Layer

SQLiteStorage: 18-column schema, FTS5 for BM25, INSERT OR IGNORE (idempotent)
ChromaDBStorage: cosine similarity, metadata pre-filtering (date_int, score), purge_stale_for_date() for freshness/TTL
DatabaseManager: facade pattern, atomic dual-write with freshness purge

Package 4: retrieval/ -- Deterministic Candidate Generation

QueryClassifier.classify(): regex intent detection (STRUCTURED/SEMANTIC/HYBRID/FINANCIAL/COMPOUND), extracts symbols, date, score, themes
BM25Search: SQLite FTS5 with porter stemming, auto-synced via triggers
HybridSearch: RRF fusion -- score(d) = 1/(k+rank), k=60
Reranker: 4 boost components (keyword 40%, EP score 30%, recency 20%, diversity 10%)
RetrievalBroker: routes query, enriches from SQL, applies reranker, builds PipelineTrace

Package 5: financial/ -- PDF Extraction Pipeline

PDFDownloader: urllib + browser headers + retry + 10MB cap
FinancialExtractor: Claude Opus 4.7, base64 PDF, JSON schema output (revenue, EBITDA, PAT, growth%, EPS, dividend, highlights, guidance)
PDFAgent: orchestrator - query SQL > download > extract > store, 1.5s delay

Package 6: agent/ -- Two AI Agents

EPAgent (tool-use loop): Claude Haiku + 4 tools, up to 10 iterations, 4-6 API calls
RetrievalAgent (candidate-first): deterministic retrieval > single LLM explain, 1 API call, 3x cheaper, per-stock evidence breakdown

5. Scoring System (Dispatch Table Pattern)

The ScoringEngine uses a dispatch table mapping each subject type to a handler method:

```
_DISPATCH = {
    "Bagging/Receiving of orders":    self._orders,
    "Outcome of Board Meeting":       self._board_meeting,
    "Acquisition":                    self._acquisition,
    "Dividend":                       self._dividend,
    "Commencement of production":     self._commissioning,
    ...  # 15 handlers total
}
```

Score	Meaning	Example
13	Order >= Rs.1000 crore	TEXRAIL Rs.4045cr railway order
11	Order Rs.100-999 crore	GOODLUCK Rs.255cr defence order
10	Order win (value not disclosed)	HFCL, RAILTEL, RVNL orders
9	Acquisition / Commissioning	WIPRO acquisition, ACMESOLAR commissioning
8	Board results / Capacity addition	DLF Q4 results, SRF capacity addition
6-7	Dividend / C-suite / Press release	LUPIN Rs.18/share dividend
4-5	Low-value dividend / Generic	Filtered but stored for completeness

6. Enrichment Examples

Every announcement gets enriched with structured metadata (all regex, zero LLM):

Symbol	Order Val	Size	Catalyst	Sector	GOV	EXP	Novel
TEXRAIL	Rs.4045cr	mega	order_win	railway,capital	No	Yes	Yes
AFCONS	Rs.2450cr	mega	order_win	infrastructure	No	No	Yes
GOODLUCK	Rs.255cr	medium	order_win	defence	No	No	Yes
HFCL	Rs.162cr	medium	order_win	infrastructure	No	Yes	Yes
NIBE	Rs.156cr	medium	order_win	defence	Yes	No	Yes
BIOCON	Rs.4569cr	mega	results	pharma	No	No	Yes
LUPIN	n/a	unknown	results	pharma	No	No	Yes
ACMESOLAR	n/a	unknown	commissioning	solar	No	No	Yes

7. Retrieval Pipeline (with Example)

Query: 'defence stocks with large orders'

Step 1 - QueryClassifier (pure Python, no LLM):

```
ParsedQuery(
    intent     = SEMANTIC (vector_only)
    themes     = ['defence']
    symbols    = []
    date_filter= None
    min_score  = None
)
```

Step 2 - RetrievalBroker routes to vector search (because intent=SEMANTIC):

```
ChromaDB semantic search --> 40 hits in 541ms
Top vector hits:
  #1 BLKASHYAP  Bagging/Receiving of orders  dist=0.621
  #2 PREMEXPLN  Bagging/Receiving of orders  dist=0.674
  #3 AVANTEL    Bagging/Receiving of orders  dist=0.693
```

Step 3 - RRF Fusion (BM25 + Vector merged):

```
score(doc) = sum_over_lists( 1 / (k + rank) )  where k=60
Fused pool: 20 candidates with combined RRF scores
```

Step 4 - Proxy Reranker (4 components):

```
final = base_rrf * (1 + kw_boost + score_boost + recency_boost + diversity)

Components:
  keyword_boost  = 0.4 * jaccard(query_tokens, doc_tokens)
  score_boost    = 0.3 * (ep_score / 13)
  recency_boost  = 0.2 * max(0, 1 - age_days/7)
  diversity_boost= 0.1 if doc in both BM25 and vector
```

Step 5 - Per-stock evidence (sent to LLM):

```
TEXRAIL [13]  order=Rs.4045cr(mega) | catalyst=order_win
              sectors=[capital_goods,railway] | EXPORT
              src=sql | rrf=1.300 | boosts(kw=0.000,sc=0.300)

GOODLUCK [11] order=Rs.255cr(medium) | catalyst=order_win
              sectors=[defence]
              src=sql | rrf=1.070 | boosts(kw=0.011,sc=0.254)
```

Step 6 - Single Claude Haiku call explains the ranked candidates with 'WHY INCLUDED' and 'CONCERN' per stock, ending with 'EP Watch' summary.

8. Pipeline Trace (Full Observability)

Every query produces a PipelineTrace logging all 6 stages:

```
----------------------------------------------------------------------
  PIPELINE TRACE  |  'order crore score above 10'  |  total 15ms
----------------------------------------------------------------------

[CLASSIFIER  ]  hits=1   0.0ms  intent=sql_only  themes=[]
                                symbols=[]  min_score=10

[SQL         ]  hits=21  0.5ms  filters: score>=10
  #1 [13] KEC          Bagging/Receiving of orders   score=1.00
  #2 [13] AFCONS       Bagging/Receiving of orders   score=1.00
  #3 [11] BLKASHYAP    Bagging/Receiving of orders   score=0.85

[FUSED       ]  hits=21  0.0ms  pre-rerank pool=21

[RERANKED    ]  hits=5   0.0ms  query_tokens=5
  #1 [13] AFCONS       rrf=1.318  src=sql
  #2 [13] KEC          rrf=1.300  src=sql
  #3 [11] EMSLIMITED   rrf=1.075  src=sql
----------------------------------------------------------------------
```

Render modes: trace.render('full') | trace.render('compact') | trace.render('json')
CLI: python main.py ask2 --trace or --trace-only (no LLM call)

9. Two AI Agents Compared

Aspect	Agent 1: EPAgent	Agent 2: RetrievalAgent
Architecture	Tool-use loop (up to 10 rounds)	Candidate-first + single explain
Model	Claude Haiku 4.5	Claude Haiku 4.5
API calls/query	4-6 calls	1 call
Cost	Higher (multi-round)	~3x cheaper
Tools available	query_sql, semantic_search, etc.	None (deterministic retrieval)
Prompt caching	Yes (5-min TTL)	Yes (5-min TTL)
Observability	Tool call log	Full PipelineTrace
Per-stock evidence	No	Yes (boost breakdown)

10. Input / Output Examples

Example 1: Ingest Command

```
$ python main.py ingest CF-AN-equities-16-06-2026-to-23-06-2026.csv

Ingesting: CF-AN-equities-16-06-2026-to-23-06-2026.csv  (min_score=4)
Screener: 223 candidates passed filtering.
Saved -> SQL: 223 new rows | ChromaDB: 223 upserted

Top candidates from this batch:
  [11]  GOODLUCK   medium  NEW  | defence
  [11]  TEXRAIL    medium  NEW  | capital_goods,railway
  [10]  MARINE     unknown NEW  | order/contract
  [10]  HFCL       unknown NEW  | railway
```

Example 2: Top EP Candidates

```
$ python main.py top --min-score 10 --limit 5

  TOP EP CANDIDATES  (score >= 10)
  [13] ##############  TEXRAIL     Texmaco Rail & Engineering
       2026-05-12  |  order/contract, Rs.4045 cr, railway
  [13] ##############  AFCONS      Afcons Infrastructure
       2026-05-04  |  order/contract, Rs.2450 cr, infrastructure
  [13] ##############  KEC         KEC International
       2026-05-05  |  order/contract, Rs.1002 cr, power
```

Example 3: Semantic Search

```
$ python main.py search "solar renewable capacity expansion"

  SEMANTIC SEARCH: "solar renewable capacity expansion"
  [9] ACMESOLAR   Commencement of commercial production
      similarity distance: 0.5734
  [9] PREMIERENE  Acquisition
      similarity distance: 0.6312
  [8] BORORENEW   Outcome of Board Meeting
      similarity distance: 0.6589
```

Example 4: AI Agent Query

```
$ python main.py ask2 "Which defence stocks received large orders?"

Intent: hybrid | Candidates: 5 | Pipeline: 560ms

TEXRAIL [13] - Rs.4045cr mega railway order with export
  component. Largest order in the database. Strong EP setup.
GOODLUCK [11] - Rs.255cr defence order. Mid-cap defence
  play, respectable order size for company revenue.
NIBE [11] - Rs.156cr defence order via subsidiary.
  Government/PSU contract. Interesting defence angle.

EP Watch: TEXRAIL stands out with mega Rs.4045cr order -
  railway + export + multiple follow-on orders in same week.
```

11. Design Patterns Used

Pattern	Where	Benefit
Facade	DatabaseManager over SQL+ChromaDB	Single entry point, keeps DBs in sync
Strategy	RetrievalBroker routes queries	Easy to add new retrieval strategies
Dispatch Table	ScoringEngine (15 handlers)	Clean separation per subject type
Abstract Base	BaseStorage for SQL/ChromaDB	Swappable backends (Postgres, Qdrant)
Pipeline/Chain	CSV>filter>score>enrich>store	Each stage independently testable
Idempotent Write	INSERT OR IGNORE + upsert	Safe to re-ingest same CSV
Migration	ALTER TABLE for new columns	Backward compat with older DBs
Prompt Caching	cache_control: ephemeral	~90% token cost savings

12. Cost Optimizations

Technique	Where	Savings
Subject gate first	AnnouncementFilter	Drops 80% before scoring
Compiled regexes	FilterConfig (frozen)	Zero re-compilation per row
Prompt caching	Agent system prompts	~90% token cost reduction
Haiku for agent	EPAgent + RetrievalAgent	$1/M vs $15/M (Opus)
Deterministic retrieval	Retrieval package	Candidate gen costs $0
Single LLM call	RetrievalAgent (Agent 2)	3x cheaper than tool loop
FTS5 built-in	BM25Search	No extra search server

13. Key Numbers

- 36 Python files, ~5,000 lines of code
- 7 packages: screener, enrichment, storage, retrieval, financial, agent, CLI
- 1,268 announcements processed (May 4 - June 23, 2026)
- 18-column SQL schema with 7 enrichment columns
- 13 sector labels, 7 catalyst types, 5 size buckets
- 2 AI agents (tool-loop + candidate-first)
- 3 search methods: SQL exact, BM25 keyword (FTS5), vector semantic (ChromaDB)
- RRF fusion merges BM25 + vector with k=60
- 4-component proxy reranker (no extra model)
- 6-stage pipeline trace for full observability
- Prompt caching: 5-min TTL, ~90% cost savings on repeated queries

14. Production Upgrade Path

Component	Current (Local)	Production
SQL Database	SQLite	PostgreSQL (change conn string only)
Vector Store	ChromaDB (local)	Qdrant (Docker/cloud)
Agent Model	Claude Haiku 4.5	Claude Opus (better accuracy)
PDF Downloads	urllib (sequential)	aiohttp (async parallel)
Reranker	Proxy (Jaccard)	Cross-encoder model
Deployment	CLI	FastAPI + Scheduler + Dashboard

15. CLI Commands Reference

Command	Description
ingest <csv>	Parse, score, enrich, store in SQL + ChromaDB
ask '<question>'	Agent 1: tool-use loop (Claude Haiku)
ask2 '<question>'	Agent 2: deterministic retrieval + single LLM explain
ask2 --trace-only	Show full pipeline trace without calling LLM
enrich	Backfill sector/materiality/novelty on existing DB rows
top --min-score N	Top EP candidates directly from SQL (no AI)
search '<query>'	Semantic similarity search via ChromaDB
extract --limit N	Download PDFs + extract financials via Claude Opus
financials	View extracted revenue/PAT/EPS table
stats	Database row counts and top symbols

Built with Python 3.12 | Claude AI (Anthropic) | SQLite + ChromaDB